

DAILY TASK 16

NAME : SANJEEV

DATE : 28-08-2025

- Aot by Attack on Titan with rating of 8.5 which costs around ₹1250
- Investor by trading book with rating of 6.8 which costs around ₹320
- Moby Dick by Adventure with rating of 9.1 which costs around ₹500
- War and Peace by Epic Novel with rating of 5.5 which costs around ₹120
- The Alchemist by Inspirational with rating of 7 which costs around ₹280
- The Da Vinci by Mystery with rating of 6.2 which costs around ₹340
- When Life gives you Tangeriens by Chinese Love with rating of 8 which costs around ₹450
- JJK Kaisen by Jujutsu kaisen with rating of 8.1 which costs around ₹200
- Naruto and sasuke by Tale of a ninja warrior with rating of 10 which costs around ₹0

- Aot by Attack on Titan with rating of 8.5 which costs around ₹1250
- Investor by trading book with rating of 6.8 which costs around ₹320
- Moby Dick by Adventure with rating of 9.1 which costs around ₹500
- War and Peace by Epic Novel with rating of 5.5 which costs around ₹120
- The Alchemist by Inspirational with rating of 7 which costs around ₹280
- The Da Vinci by Mystery with rating of 6.2 which costs around ₹340
- When Life gives you Tangeriens by Chinese Love with rating of 8 which costs around ₹450
- JJK Kaisen by Jujutsu kaisen with rating of 8.1 which costs around ₹200

```
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { Book } from 'src/models/books.interface';
import { entry } from 'src/models/entry.interface';
import { NewBook } from 'src/models/newBook.interface';
import { place } from 'src/models/place.interface';
@Injectable({
  providedIn: 'root'
})
export class BookService {

  private apiUrl = "http://localhost:5056/Book";
```

```

private Login = "http://localhost:5056/User";
private token: string = "";

login(username: string = "sanjeev", password: string = "sanjeev") {
    return this.http.post(`${this.Login}/Login`, { name: username, password
}, { responseType: 'text' });
}

setToken(token: string) {
    this.token = token;
}

constructor(private http : HttpClient) { }

private getAuthHeaders() {
    return { headers: new HttpHeaders({
        'Authorization': `Bearer ${this.token}`,
        'Content-Type': 'application/json'
    })
    };
}

addbook(entry : entry)
{
    return
this.http.post<entry>(`${this.apiUrl}/AddBooks`,entry,this.getAuthHeaders(
))
}

getfavbook() {
    return this.http.get<Book[]>(`${this.apiUrl}/GetallBooks`,
this.getAuthHeaders());
}

getAllBooks(): Observable<Book[]> {
    return this.http.get<Book[]>(`${this.apiUrl}/GetallBooks`);
}

deleteBook(bookId: number) {
    return this.http.delete(`${this.apiUrl}/DeleteBook`, {

```

```

        params: { id: bookId.toString() },
        headers: new HttpHeaders({
            'Authorization': `Bearer ${this.token}`
        }),
        responseType: 'text'
    });
}

updateBook(bookId: number, book: Book) {
    return this.http.put<Book>(`${this.apiUrl}/UpdateBook/${bookId}`,
book, this.getAuthHeaders());
}

getAllNewBooks() {
    return this.http.get<NewBook[]>(`${this.apiUrl}/FetchAllNewBooks`,
this.getAuthHeaders());
}

getfavplaces(): place[]
{
    return [
        {Name : 'Valparai', altitude: 1800},
        {Name : 'Munnar', altitude: 2000},
        {Name : 'Dhimbam', altitude: 1500}
    ]
}
}

```

```

import { Component } from '@angular/core';
import { place } from 'src/models/place.interface';
import { BookService } from '../book.service';
import { Book } from 'src/models/books.interface';

```

```

import { entry } from 'src/models/entry.interface';
import { NewBook } from 'src/models/newBook.interface';
@Component({
  selector: 'app-test',
  templateUrl: './test.component.html',
  styleUrls: ['./test.component.css']
})

export class TestComponent {
  constructor(private bookService: BookService)
  {

  }

  ooty:place = {Name:'Leh Ladakh', altitude: 2240}
  newPlace : string = ""
  altplace : number = 0
  favBook: Book[] = [];
  newBookTitle: string = "";
  newBookDescription: string = "";
  newBookRating: number = 0;
  newBookPrice: number = 0;
  deleteBookId: number = 0;
  allBooks: Book[] = [];
  selectedBook?: Book;

  addBook() {
    console.log("Started");

    const newEntry: entry = {
      title: this.newBookTitle,
      description: this.newBookDescription,
      rating: this.newBookRating,
      price: this.newBookPrice
    };

    this.bookService.addbook(newEntry).subscribe((res) => {
      this.favBook = [...this.favBook, newEntry];
    });
  }
}

```

```

    }

    );
    this.loadBooks();
}

selectBook(book: Book) {
    console.log("tetvdvchduahjebud")
    this.selectedBook = { ...book };
}

updateBook() {
    if (!this.selectedBook || this.selectedBook.bookId === undefined)
    return;

    this.bookService.updateBook(this.selectedBook.bookId, this.selectedBook)
        .subscribe(res => {
            console.log('Book updated successfully', res);
            this.loadBooks();
            this.selectedBook = undefined;
        });
}

loadBooks() {
    this.bookService.getfavbook().subscribe((res: Book[]) => {
        this.favBook = res;
        this.allBooks = res;
    });
}

addmore()
{
    console.log("clicked")
    this.my_fav_places.push({Name:'Ooty', altitude: 1200})
    this.my_fav_places = [...this.my_fav_places,{Name : this.newPlace,
altitude : this.altplace}]

```

```

    }
    my_fav_places: place[] = [
      {Name : 'Valparai', altitude: 1800},
      {Name : 'Munnar', altitude: 2000},
      {Name : 'Dhimbam', altitude: 1500}
    ]
    login() {
      this.bookService.login();
    }

    loginAndLoadBooks() {
      this.bookService.login().subscribe(token => {
        console.log("Received token:", token);
        localStorage.setItem('token', token);
        this.bookService.setToken(token);
        this.loadBooks();
      });
    }

    deleteBook() {
      if (this.deleteBookId) {
        this.bookService.deleteBook(this.deleteBookId).subscribe({
          next: () => {
            console.log(`Book with ID ${this.deleteBookId} deleted`);
            this.loadBooks();
            this.deleteBookId = 0;
          }
        });
      }
    }

    loadAllBooks() {
      this.bookService.getAllNewBooks().subscribe(({data: NewBook[]}) => {
        // this.allBooks = data;
        console.log("Fetched nested books:", this.allBooks);
      });
    }

```

```

place_fr_ser :string = ""
my_place:place[] = []
// ngOnInit()
// {
//   // this.my_place = this.bookService.getfavplaces()
//   // console.log( `init ${this.my_place}`)

// console.log("before")
// this.favBook = this.bookService.getfavbook();
//   console.log("favBook from service:", this.favBook);
// }

ngOnInit() {
//   console.log("before");

//   this.bookService.getfavbook().subscribe((res: Book[]) => {
//     this.favBook = res;
//     console.log("favBook :", this.favBook);
//   });
this.loadBooks();
this.loadAllBooks();
}

ngAfterContentInit()
{
  console.log("After content Initialized")
}
}

```

```

<input id="nameInput" type="text" [(ngModel)]="newPlace"
placeholder="Place Name">
<input id="altInput" type="number" [(ngModel)]="altplace"
placeholder="Altitude">
<button (click)="addmore()">Add place</button>

```

```
<ul>
  <li *ngFor="let a of my_fav_places">
    {{ a.Name }} is at {{ a.altitude }} m
  </li>
</ul>

<div>
  <input type="text" [(ngModel)]="newBookTitle" placeholder="Title">
  <input type="text" [(ngModel)]="newBookDescription"
placeholder="Description">
  <input type="number" [(ngModel)]="newBookRating" placeholder="Rating">
  <input type="number" [(ngModel)]="newBookPrice" placeholder="Price">
  <button (click)="addBook()">Add Book</button>
</div>

<div>
  <input type="number" [(ngModel)]="deleteBookId" placeholder="Enter Book
ID to delete">
  <button (click)="deleteBook()">Delete Book</button>
</div>
```